

EMBEDDED SYSTEMS REPORT

Arduino UNO Sensor Experiments

Name	Mukesh
Roll Number	23BTT309
Team	Alpha Team 1
Subject	Embedded Systems
Platform	Arduino UNO
Sensors Covered	IR Sensor, Ultrasonic Sensor, DHT11
Date	06 April 2026

Prepared as part of Embedded Systems Laboratory Practice

1. Introduction

This report documents the hands-on experiments conducted using the Arduino UNO microcontroller board as part of the Embedded Systems laboratory practice. Three different types of sensors were interfaced with the Arduino UNO to understand real-world sensing, data acquisition, and control logic implementation.

The Arduino UNO is an open-source microcontroller board based on the ATmega328P chip, offering 14 digital I/O pins, 6 analog input pins, and a USB interface for programming and serial communication. It is widely used in embedded systems prototyping due to its simplicity and versatility.

The experiments performed are:

1. IR (Infrared) Sensor – Object Detection
2. Ultrasonic Sensor (HC-SR04) – Distance Measurement
3. DHT11 Sensor – Temperature & Humidity Monitoring

2. Experiment 1 – IR Sensor

2.1 Aim

To detect the presence or absence of an object using an IR (Infrared) sensor and control an LED output accordingly using Arduino UNO.

2.2 Description

An Infrared (IR) sensor module works by emitting infrared light from an IR LED and detecting the reflected light using a photodiode or phototransistor. When an object is placed in front of the sensor, the emitted IR beam is reflected back and the sensor outputs a LOW signal on its OUT pin. When no object is present, the output is HIGH.

In this experiment, the OUT pin of the IR sensor is connected to digital pin 2 of the Arduino. An LED is connected to digital pin 13. The Arduino reads the sensor state and turns the LED ON when no object is detected and turns it OFF when an object is detected. The results are also printed via the Serial Monitor.

2.3 Pin Configuration

IR Sensor Pin	Arduino Pin	Purpose
VCC	5V	Power Supply
GND	GND	Ground
OUT	Digital Pin 2	Sensor Output
LED	Digital Pin 13	Output Indicator

2.4 Arduino Code

```
const int irSensorPin = 2;    // IR sensor OUT pin
const int ledPin = 13;        // LED pin

void setup() {
  pinMode(irSensorPin, INPUT);
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int sensorValue = digitalRead(irSensorPin);
  if (sensorValue == LOW) {
    // Object detected
    digitalWrite(ledPin, LOW);    // Turn OFF LED
    Serial.println("Object Detected - LED OFF");
  } else {
    // No object
    digitalWrite(ledPin, HIGH);  // Turn ON LED
    Serial.println("No Object - LED ON");
  }
}
```

2.5 Experimental Setup

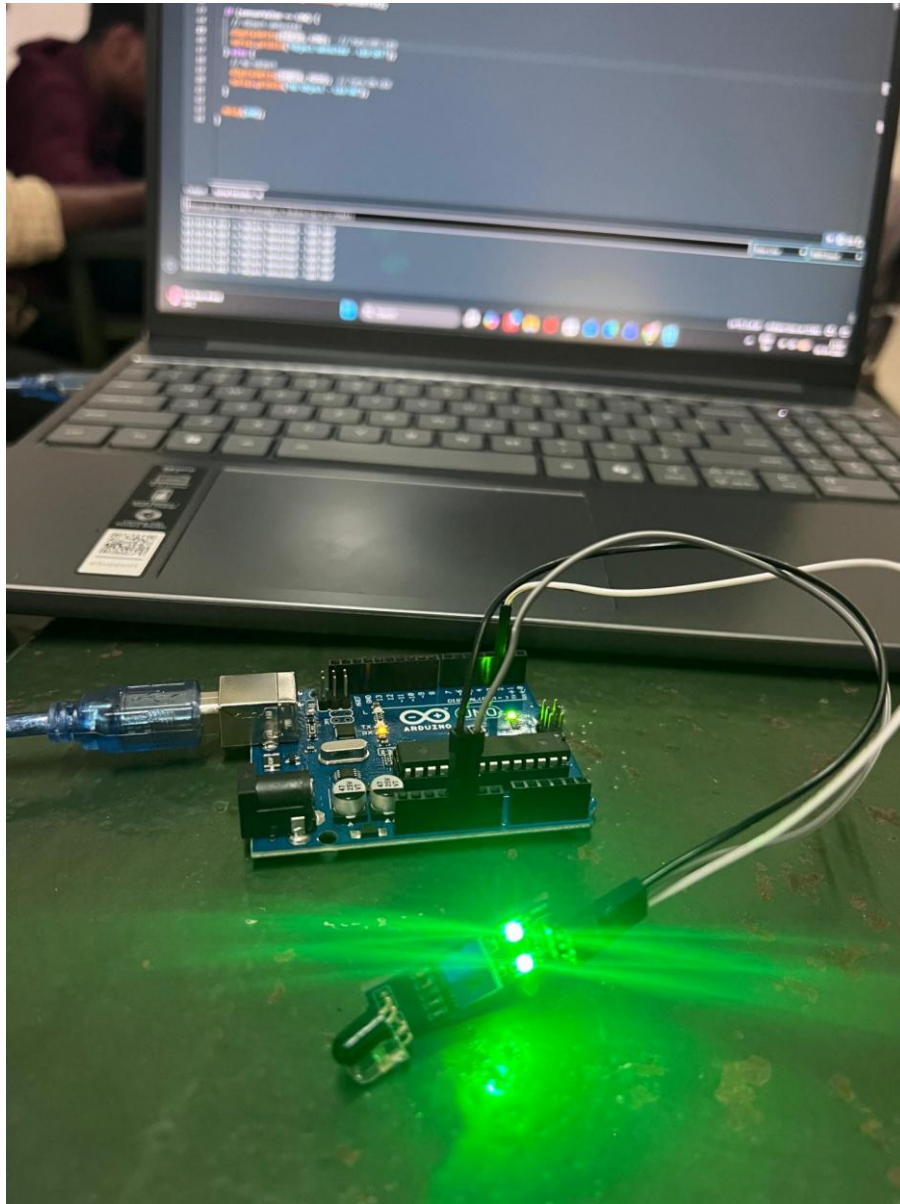


Figure 1: IR Sensor connected to Arduino UNO with LED indicators

2.6 Output / Observation

When an object is brought close to the IR sensor, the LED connected to Pin 13 turns OFF and the Serial Monitor prints "Object Detected - LED OFF". When the object is removed, the LED turns back ON and the Serial Monitor prints "No Object - LED ON". The green LED visible in the image confirms the circuit is active and functioning correctly.

3. Experiment 2 – Ultrasonic Sensor

3.1 Aim

To measure the distance of an object using the HC-SR04 Ultrasonic Sensor and display the measured distance over the Serial Monitor, while controlling an LED based on proximity.

3.2 Description

The HC-SR04 Ultrasonic Sensor uses sound waves to measure distance. It has four pins: VCC, GND, TRIG (Trigger), and ECHO. The microcontroller sends a 10-microsecond pulse to the TRIG pin, which causes the sensor to emit an ultrasonic burst. The ECHO pin then goes HIGH for a duration proportional to the time taken for the sound to travel to the object and return.

The distance is calculated using the formula: $\text{Distance (cm)} = (\text{Duration} \times 0.034) / 2$, where 0.034 cm/microsecond is the speed of sound and dividing by 2 accounts for the round trip. In this experiment, the LED turns OFF when an object is detected within 5 cm, and ON when the object is farther away.

3.3 Pin Configuration

HC-SR04 Pin	Arduino Pin	Purpose
VCC	5V	Power Supply
GND	GND	Ground
TRIG	Digital Pin 9	Trigger Output
ECHO	Digital Pin 10	Echo Input

3.4 Arduino Code

```
const int trigPin = 9;
const int echoPin = 10;
const int ledPin = 13;
long duration;
int distance;

void setup() {
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  duration = pulseIn(echoPin, HIGH);
```

```
Serial.print("Distance: ");  
Serial.print(distance);  
Serial.println(" cm");  
if (distance < 5) {  
    digitalWrite(ledPin, LOW);    // LED OFF  
} else {  
    digitalWrite(ledPin, HIGH);   // LED ON  
}  
delay(200);  
}
```

3.5 Experimental Setup

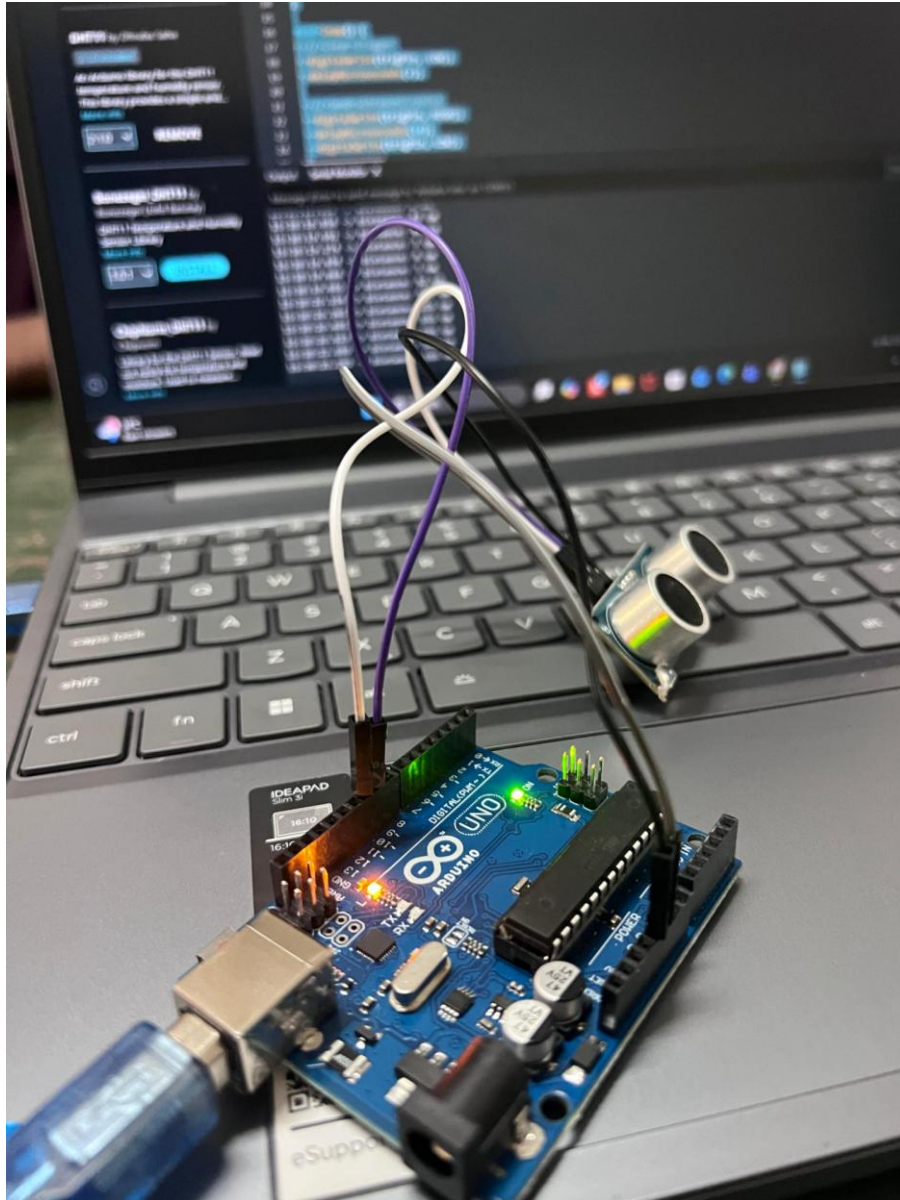


Figure 2: HC-SR04 Ultrasonic Sensor connected to Arduino UNO

3.6 Output / Observation

The Serial Monitor displays real-time distance readings in centimeters (e.g., "Distance: 12 cm"). When an object is closer than 5 cm to the sensor, the LED turns OFF, simulating a proximity

alert. Beyond 5 cm, the LED remains ON. The HC-SR04 sensor is visible in the photo, connected to the Arduino via white, black, and purple jumper wires.

4. Experiment 3 – DHT11 Temperature & Humidity Sensor

4.1 Aim

To read the ambient temperature (in degrees Celsius) and relative humidity (in percentage) using the DHT11 sensor and display the values via the Arduino Serial Monitor.

4.2 Description

The DHT11 is a basic digital temperature and humidity sensor. It uses a capacitive humidity sensor and a thermistor to measure the surrounding air. It outputs data as a single-wire digital signal. The sensor operates on 3.3V–5V power and communicates using a custom single-wire protocol.

The DHT library (by Adafruit) is used in this experiment to simplify reading from the sensor. The sensor is connected to digital pin 2 on the Arduino. A minimum 2-second delay between readings is required as specified by the DHT11 datasheet. If the reading fails (returns NaN), an error message is printed to the Serial Monitor.

4.3 Pin Configuration

DHT11 Pin	Arduino Pin	Purpose
VCC	5V	Power Supply
GND	GND	Ground
DATA	Digital Pin 2	Data Signal

4.4 Arduino Code

```
#include <DHT.h>

#define DHTPIN 2
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  dht.begin();
}

void loop() {
  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();
  if (isnan(humidity) || isnan(temperature)) {
    Serial.println("Failed to read from DHT sensor!");
    return;
  }
  Serial.print("Humidity: ");
  Serial.print(humidity);
  Serial.print(" % | ");
  Serial.print("Temperature: ");
```



```
Serial.print(temperature);  
Serial.println(" degrees C");  
delay(2000);  
}
```

4.5 Experimental Setup

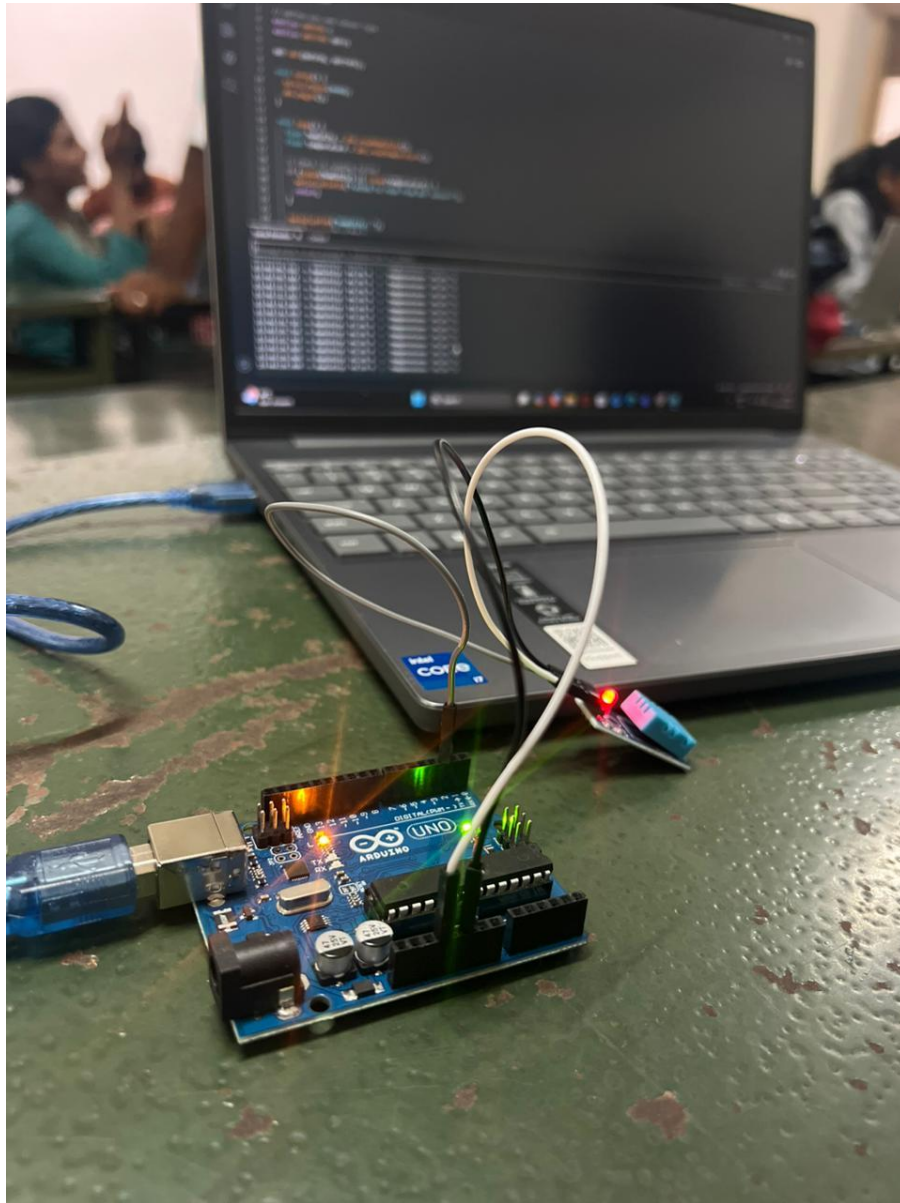


Figure 3: DHT11 Sensor connected to Arduino UNO

4.6 Output / Observation

The Serial Monitor prints temperature and humidity readings every 2 seconds in the format: "Humidity: 65 % | Temperature: 28 degrees C". The red LED on the DHT11 module indicates the sensor is powered and active. The Arduino IDE Serial Monitor displayed consistent readings, confirming successful data acquisition from the sensor.

5. Conclusion

All three sensor experiments were successfully implemented using the Arduino UNO microcontroller. The following key learnings were obtained:

- IR Sensor: Object detection using reflected infrared light and controlling digital output pins based on sensor state.
- Ultrasonic Sensor: Distance measurement using time-of-flight of sound waves (pulseIn function) and deriving physical distance with the speed-of-sound formula.
- DHT11 Sensor: Interfacing a library-based digital sensor for environmental data acquisition (temperature and humidity) with built-in error handling for failed reads.

These experiments demonstrate fundamental concepts of embedded systems including digital I/O, serial communication, sensor interfacing, and real-time data acquisition. The Arduino UNO platform proved to be an effective and accessible tool for learning hardware-software integration.